OOP Interview Questions and Answers

Last Updated : 14 Oct, 2025

Object-Oriented Programming (OOP) is a core paradigm in software development that models real-world entities into objects combining data and behavior. The following are topic wise interview questions on Object Oriented Programming.

1. What is Object Oriented Programming (OOPs)?

Object Oriented Programming is a programming paradigm where the complete software operates as a bunch of objects talking to each other. An object is a collection of data and the methods which operate on that data.

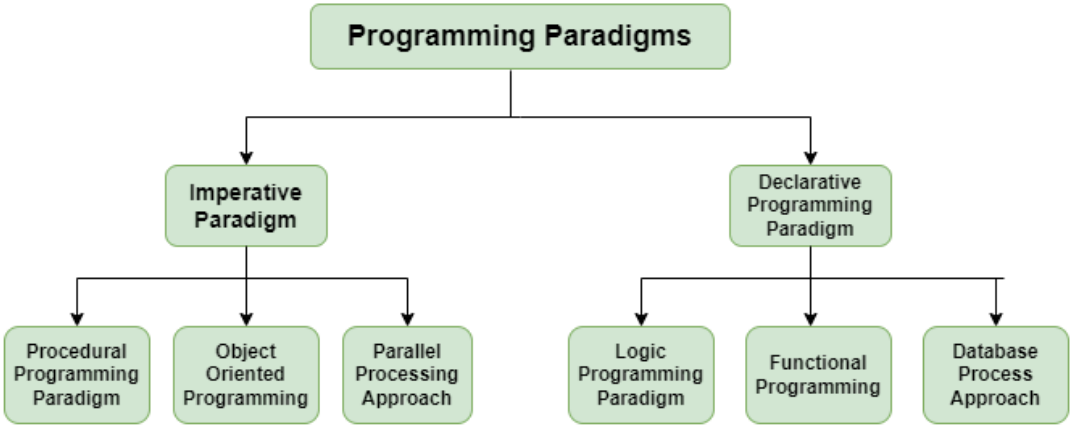
2. Why OOPs?

The main advantage of OOP is better manageable code that covers the following:

1. The overall understanding of the software is increased as the distance between the language spoken by developers and that spoken by users.
2. Object orientation eases maintenance by the use of encapsulation. One can easily change the underlying representation by keeping the methods the same.
3. The OOPs paradigm is mainly useful for relatively big software.

3. What other paradigms of programming exist besides OOPs?

The programming paradigm is referred to the technique or approach of writing a program. The programming paradigms can be classified into the following types:



Programming Paradigms

1. Imperative Programming Paradigm

It is a programming paradigm that works by changing the program state through assignment statements. The main focus in this paradigm is on how to achieve the goal. The following programming paradigms come under this category:

1. **Procedural Programming Paradigm**: This programming paradigm is based on the procedure call concept. Procedures, also known as routines or functions are the basic building blocks of a program in this paradigm.
2. **Object-Oriented Programming or OOP**: In this paradigm, we visualize every entity as an object and try to structure the program based on the state and behavior of that object.
3. **Parallel Programming**: The parallel programming paradigm is the processing of instructions by dividing them into multiple smaller parts and executing them concurrently.

2. Declarative Programming Paradigm

Declarative programming focuses on what is to be executed rather than how it should be executed. In this paradigm, we express the logic of a computation without considering its control flow. The declarative paradigm can be further classified into:

1. **Logical Programming Paradigm**: It is based on formal logic where the program statements express the facts and rules about the problem in the logical form.
2. **Functional Programming Paradigm**: Programs are created by applying and composing functions in this paradigm.
3. **Database Programming Paradigm**: To manage data and information organized as fields, records, and files, database programming models are utilized.

4. What is the difference between Structured Programming and Object-Oriented Programming?

Structured Programming is a technique that is considered a precursor to OOP and usually consists of well-structured and separated modules. It is a subset of procedural programming. The difference between OOPs and Structured Programming is as follows:

Object-Oriented Programming	Structural Programming
Programming that is object-oriented is built on objects having a state and behavior.	A program's logical structure is provided by structural programming, which divides programs into their corresponding functions.
It follows a bottom-to-top approach.	It follows a Top-to-Down approach.

Object-Oriented Programming	Structural Programming
Restricts the open flow of data to authorized parts only providing better data security.	No restriction to the flow of data. Anyone can access the data.
Enhanced code reusability due to the concepts of polymorphism and inheritance.	Code reusability is achieved by using functions and loops.
Methods work dynamically, making calls based on object behavior and the need of the code at runtime.	Functions are called sequentially, and code lines are processed step by step.
Modifying and updating the code is easier.	Modifying the code is difficult as compared to OOPs.
Data is given more importance in OOPs.	Code is given more importance.

5. What are some commonly used Object-Oriented Programming Languages?

OOPs paradigm is one of the most popular programming paradigms. It is widely used in many popular programming languages such as:

- C++
- Java
- Python
- JavaScript
- C#
- Ruby

6. What are the advantages and disadvantages of OOPs?

Advantages of OOPs	Disadvantages of OOPs
OOPs provides enhanced code reusability.	The programmer should be well-skilled and should have excellent thinking in terms of objects as everything is treated as an object in OOPs.
The code is easier to maintain and update.	Proper planning is required because OOPs is a little bit tricky.

Advantages of OOPs	Disadvantages of OOPs
It provides better data security by restricting data access and avoiding unnecessary exposure.	OOPs concept is not suitable for all kinds of problems.
Fast to implement and easy to redesign resulting in minimizing the complexity of an overall program.	The length of the programs is much larger in comparison to the procedural approach.

7. What is a Class?

A **class** is a building block of Object-Oriented Programs. It is a user-defined data type that contains the data members and member functions that operate on the data members. It is like a blueprint or template of objects having common properties and methods.

8. What is an Object?

An **object** is an instance of a class. Data members and methods of a class cannot be used directly. We need to create an object (or instance) of the class to use them. In simple terms, they are the actual world entities that have a state and behaviour.

C++ Java Python C#

```cpp
#include <iostream>
using namespace std;

// defining class
class Student {
public:
    string name;
};

int main()
{

    // creating object
    Student student1;
    // assigning member some value
    student1.name = "Rahul";

    cout << "student1.name: " << student1.name;

    return 0;
}
```

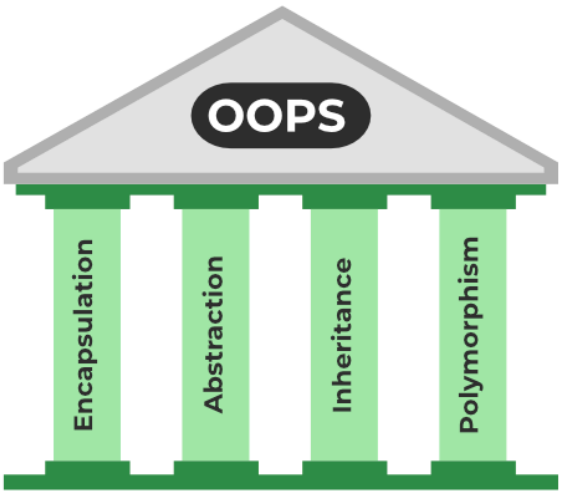
Output

```
student1.name: Rahul
```

9. What are the main features of OOPs?

The main feature of the OOPs, also known as 4 pillars or basic principles of OOPs are as follows:

1. Encapsulation
2. Data Abstraction
3. Polymorphism
4. Inheritance

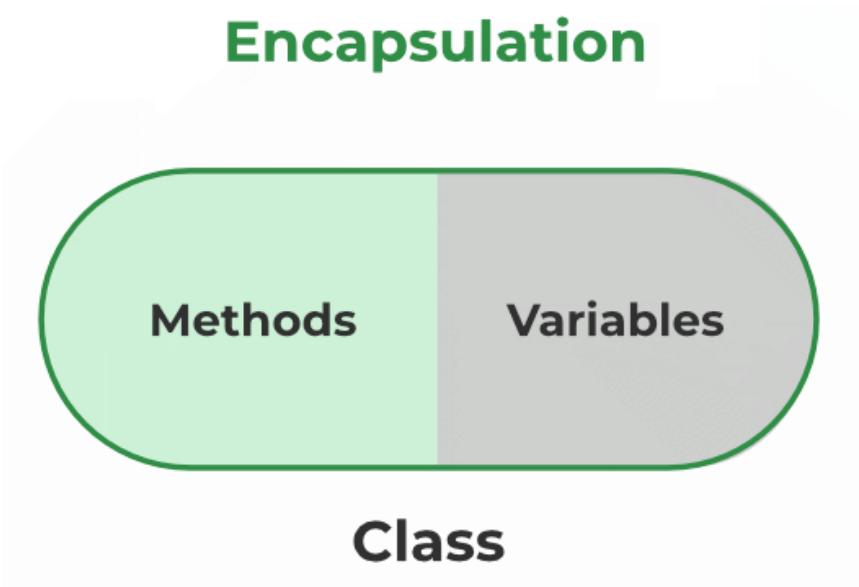


OOPs Main Features

10. What is Encapsulation?

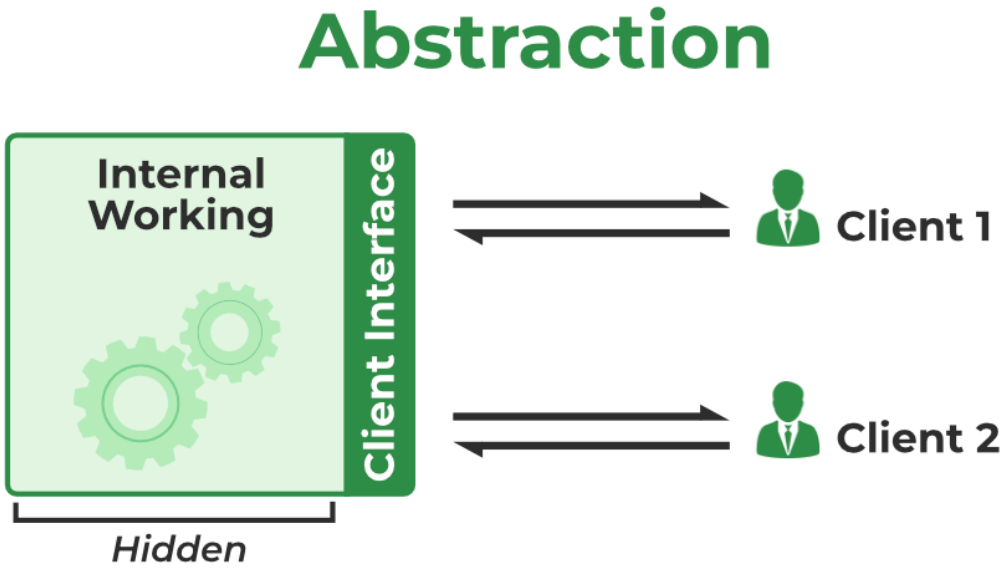
Encapsulation is the binding of data and methods that manipulate them into a single unit such that the sensitive data is hidden from the users
It is implemented as the processes mentioned below:

1. **Data hiding:** A language feature to restrict access to members of an object. For example, private and protected members in C++.
2. **Bundling of data and methods together:** Data and methods that operate on that data are bundled together. For example, the data members and member methods that operate on them are wrapped into a single unit known as a class.



11. What is Abstraction?

Abstraction is similar to data encapsulation and is very important in OOP. It means showing only the necessary information and hiding the other irrelevant information from the user. Abstraction is implemented using classes and interfaces.



12. What is Inheritance? What is its purpose?

The idea of inheritance is simple, a class is derived from another class and uses data and implementation of that other class. The class which is derived is called child or derived or subclass and the class from which the child class is derived is called parent or base or superclass.

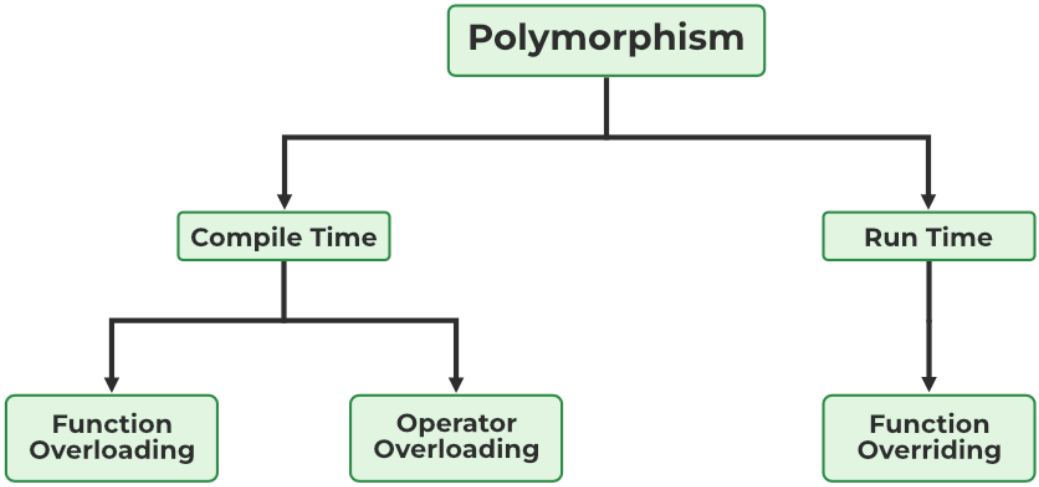
The main purpose of Inheritance is to increase code reusability. It is also used to achieve Runtime Polymorphism.

13. What is Polymorphism? and types of Polymorphism?

The word "**Polymorphism**" means having many forms. It is the property of some code to behave differently for different contexts. For example, in C++ language, we can define multiple functions having the same name but different working depending on the context.

Polymorphism can be classified into two types based on the time when the call to the object or function is resolved. They are as follows:

- Compile Time Polymorphism
- Runtime Polymorphism



Polymorphism

A) Compile-Time Polymorphism

Compile time polymorphism, also known as static polymorphism or early binding is the type of polymorphism where the binding of the call to its code is done at the compile time. Method overloading or operator overloading are examples of compile-time polymorphism.

B) Runtime Polymorphism

Also known as dynamic polymorphism or late binding, runtime polymorphism is the type of polymorphism where the actual implementation of the function is determined during the runtime or execution. Function overriding is an example of this method.

14. What are access specifiers? What is their significance in OOPs?

Access specifiers are special types of keywords that are used to specify or control the accessibility of entities like classes, methods, and so on. **Private**, **Public**, and **Protected** are examples of access specifiers or access modifiers.
The key components of OOPs, encapsulation and data hiding, are largely achieved because of these access specifiers.

15. What is the difference between overloading and overriding?

A compile-time polymorphism feature called **overloading** allows an entity to have numerous implementations of the same name. Method overloading and operator overloading are two examples.

Overriding is a form of runtime polymorphism where an entity with the same name but a different implementation is executed. It is implemented with the help of virtual functions.

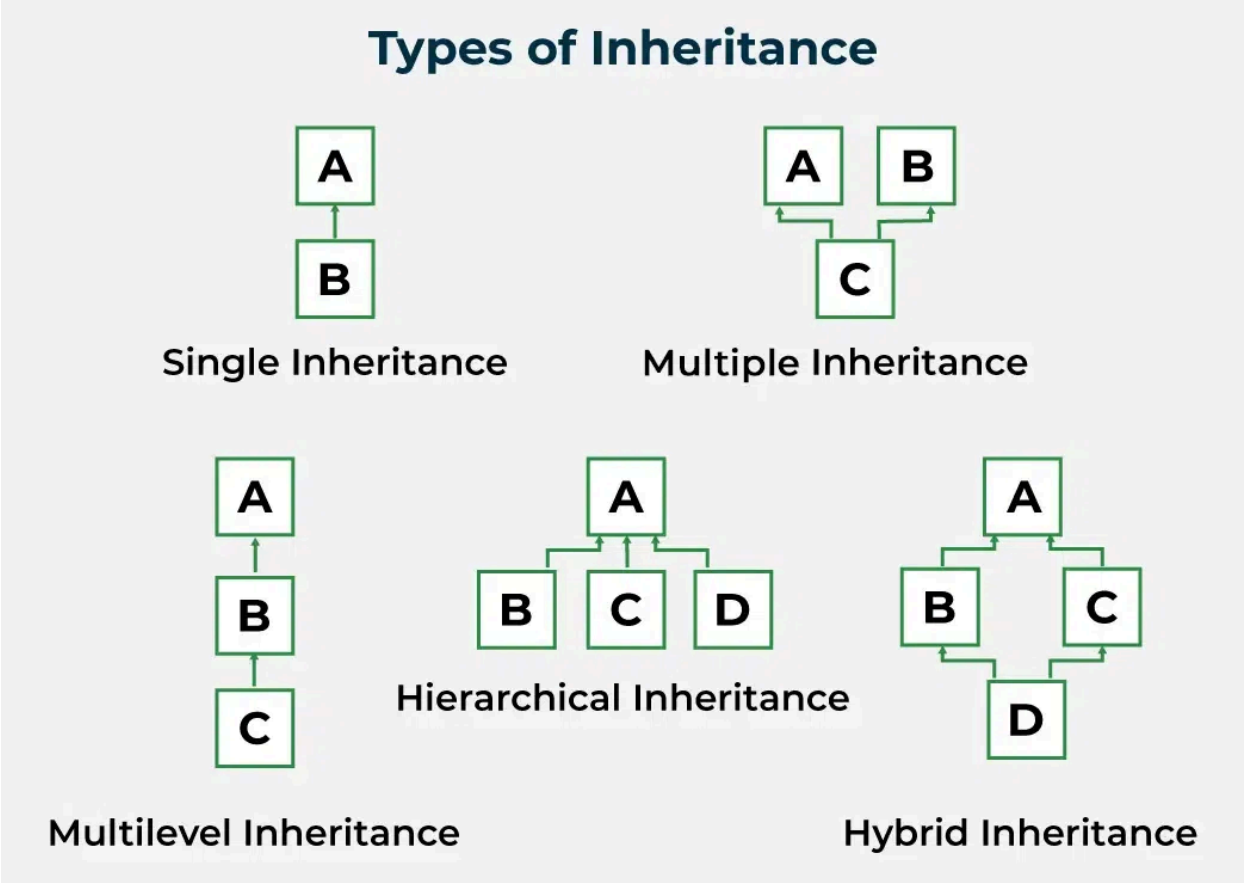
16. Are there any limitations on Inheritance?

Yes, there are more challenges when you have more authority. Although inheritance is a very strong OOPs feature, it also has significant drawbacks.

- As it must pass through several classes to be implemented, inheritance takes longer to process.
- The base class and the child class, which are both engaged in inheritance, are also closely related to one another (called tightly coupled). Therefore, if changes need to be made, they may need to be made in both classes at the same time.
- Implementing inheritance might be difficult as well. Therefore, if not implemented correctly, this could result in unforeseen mistakes or inaccurate outputs.

17. What different types of Inheritance are there?

Inheritance can be classified into 5 types which are as follows:



Types of Imheritance

1. **Single Inheritance:** Child class derived directly from the base class
2. **Multiple Inheritance:** Child class derived from multiple base classes.

3. **Multilevel Inheritance:** Child class derived from the class which is also derived from another base class.
4. **Hierarchical Inheritance:** Multiple child classes derived from a single base class.
5. **Hybrid Inheritance:** Inheritance consisting of multiple inheritance types of the above specified.

Note: *Type of inheritance supported is dependent on the language. For example, Java does not support multiple inheritance.*

18. What is an interface?

A unique class type known as an interface contains methods but not their definitions. Inside an interface, only method declaration is permitted. You cannot make objects using an interface. Instead, you must put that interface into use and specify the procedures for doing so.

19. How is an abstract class different from an interface?

Both abstract classes and interfaces are special types of classes that just include the declaration of the methods, not their implementation. An abstract class is completely distinct from an interface, though. Following are some major differences between an abstract class and an interface.

Abstract Class	Interface
A class that is abstract can have both abstract and non-abstract methods.	An interface can only have abstract methods.
An abstract class can have final, non-final, static and non-static variables.	The interface has only static and final variables.
Abstract class doesn't support multiple inheritance	An interface supports multiple inheritance.

20. How much memory does a class occupy?

Classes do not use memory. They merely serve as a template from which items are made. Now, objects actually initialize the class members and methods when they are created, using memory in the process.

21. Is it always necessary to create objects from class?

No. If the base class includes non-static methods, an object must be constructed. But no objects need to be generated if the class includes static methods. In this instance, you can

use the class name to directly call those static methods.

22. What is the difference between a structure and a class in C++?

The structure is also a user-defined datatype in C++ similar to the class with the following differences:

- The major difference between a structure and a class is that in a structure, the members are set to public by default while in a class, members are private by default.
- The other difference is that we use **struct** for declaring structure and **class** for declaring a class in C++.

23. What is Constructor?

A constructor is a block of code that initializes the newly created object. A constructor resembles an instance method but it's not a method as it doesn't have a return type. It generally is the method having the same name as the class but in some languages, it might differ. For example:

In python, a constructor is named **__init__.**

In C++ and Java, the constructor is named the same as the class name.

Example:

C++ Java Python C#

```
class base {
    public:
        base() { cout << "This is a constructor"; }
}
```

24. What are the various types of constructors in C++?

The most common classification of constructors includes:

1. **Default Constructor**
2. **Non-Parameterized Constructor**
3. **Parameterized Constructor**
4. **Copy Constructor**

1. Default Constructor

The default constructor is a constructor that doesn't take any arguments. It is a non-parameterized constructor that is automatically defined by the compiler when no explicit constructor definition is provided.

It initializes the data members to their default values.

2. Non-Parameterized Constructor

It is a user-defined constructor having no arguments or parameters.

Example:

C++ Java Python C#

```
class base {
public:
    base(){
        cout << "This is a non-parameterized contructor";
    }
}
```

3. Parameterized Constructor

The constructors that take some arguments are known as parameterized constructors.

Example:

C++ Java Python C#

```
class base {
public:
    int base;
    base(int var) {
        cout << "Constructor with argument: " << var;
    }
};
```

4. Copy Constructor

A copy constructor is a member function that initializes an object using another object of the same class.

Example:

C++ Java Python C#

```
class base {
    int a, b;

    // copy constructor
    base(base& obj) {
        a = obj.a;
        b = obj.b;
    }
}
```

In Python, we do not have built-in copy constructors like Java and C++ but we can make a workaround using different methods.

25. What is a destructor?

A destructor is a method that is automatically called when the object goes out of scope or destroyed.

In C++, the destructor name is also the same as the class name but with the (~) **tilde symbol** as the prefix.

In Python, the destructor is named **__del__**.

Example:

C++ Java Python C#

```
class base {
public:
    ~base() { cout << "This is a destructor"; }
}
```

In Java, the garbage collector automatically deletes the useless objects so there is no concept of destructor in Java. We could have used finalize() method as a workaround for the java destructor, but it is also deprecated since Java 9.

26. Can we overload the constructor in a class?

Yes We can overload the constructor in a class in Java. Constructor Overloading is done when we want constructor with different constructor with different parameter (Number and Type).

27. Can we overload the destructor in a class?

No, a destructor cannot be overloaded in a class. There can only be one destructor present in a class.

28. What are friend functions and friend classes?

Friend Functions: A friend function is a special function that is allowed to access private and protected data of a class, even though it's not a member of the class.

Friend Class: A friend class is a class that can access the private and protected members of another class. It's like allowing a trusted friend or a group of friends to see and change your personal information, which others cannot.

29. What is the virtual function and pure virtual function?

A virtual function is a function that is used to override a method of the parent class in the derived class. It is used to provide abstraction in a class.

- In C++ and C#, a virtual function is declared using the virtual keyword,
- In Java, every public, non-static, and non-final method is a virtual function.

- Python methods are always virtual.

Example:

C++ Java Python C#

```
class base {
    virtual void print() {
        cout << "This is a virtual function";
    }
}
```

A pure virtual function, also known as an abstract function, is a member function that doesn't contain any statements. This function is defined in the derived class if needed.

Example:

C++ Java C#

```
class base {
    virtual void pureVirFunc() = 0;
}
```

In Python, we achieve this using @abstractmethod from the ABC (Abstract Base Class) module.

30. What is an abstract class?

In general terms, an abstract class is a class that is intended to be used for inheritance. It cannot be instantiated. An abstract class can consist of both abstract and non-abstract methods.

- In C++, an abstract class is a class that contains at least one pure virtual function.
- In Java and C#, an abstract class is declared with an **abstract** keyword.

Example:

C++ Java C#

```
class absClass {
public:
    virtual void pvFunc() = 0;
}
```

In Python, we use ABC (Abstract Base Class) module to create an abstract class.

Topic-wise Interview Questions:

- OOP Core Principles
- Design Patterns & Best Practices

Resources:

1. Core Concepts: Classes & Objects, Encapsulation, Polymorphism, Abstraction, Access Modifiers, Inheritance

2. Advanced Topics: Creational Design Patterns, Structural Design Patterns, Behavioral Design Patterns, SOLID Principles, Refactoring Techniques.

Related Posts:

1. OOPs in C++
2. OOPs in Java
3. OOPs in Python
4. Classes and Objects in C++

7. Introduction to Programming Paradigms
8. Interface in Java
9. Abstract Class in Java
10. C++ Interview Questions

Comment GeeksforGeeks 327

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Careers
Contact Us
Corporate Solution
Campus Training

Explore
POTD
Practice Problems
Blogs
Upskill Courses
Connect

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
GATE
School Subjects
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Data Science
Programming Languages
DevOps & Cloud
GATE
MongoDB
Certifications

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

GET IT ON Google Play
Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved